

Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

Diseño de una Máquina Expendedora usando una MEF con arquitectura Moore

Teoría de la Computación

Alumno: José Luis Chávez Gómez

Codigo: 220419709

11 de noviembre del 2026

1 Definición formal

La máquina expendedora se define formalmente como el autómata:

$$M = (Q, \Sigma, \delta, q_0, F)$$

donde:

- Q **(Conjunto de estados):**
  - q_0: Estado inicial.
  - $q_{x,s}$: Estados de acumulación de saldo (x es el productoy s es el dinero).
  - $q_{entrega}$: Estado de Aceptación.
  - q_{error}: Estado de error.
- Σ **(Alfabeto):** Productos (a-j) y monedas.

  $\Sigma = \{a, b, c, d, e, f, g, h, i, j, 1, 2, 5, 10\}$.
- q_0 **(Estado inicial):** q_0
- F **(Estados de aceptación):** $F = \{q_{x,s} \mid s \geq precio(x)\}$
- δ **(Función de transición):** $Q \times \Sigma \to Q$:
  - **Selección de producto:** $\delta(q_0, x) = q_{x,0}$ (Pasa a esperar dinero).
  - **Acumulación de monedas:** $\delta(q_{x,s}, m) = q_{x,s+m}$ (Suma el valor).
  - **Error de selección previa:** $\delta(q_0, m) = q_{error}$ (No se inserta dinero antes del producto).
  - **Error de selección doble:** $\delta(q_{x,s}, y) = q_{error}$ (Solo un producto por transaccion).

2 Diseño de estados

- **Estado Inicial:**
  - q_0
- **Estados de Selección de Producto:**
  - $q_{x,0}$
- **Estados de Acumulación de Dinero:**
  - $q_{x,s}$

- **Estados de Aceptación (Entrega de producto):**
  - $q_{entrega,x}$
- **Estado de Error:**
  - q_{error}: La máquina transita hacia aquí si se violan las reglas:
    1. Insertar monedas (m) estando en q_0.
    2. Seleccionar un segundo producto.

3 Tabla de transiciones

$\delta : Q \times \Sigma \to Q$

- x: producto seleccionado ($x \in \{a, b, c, d, e, f, g, h, i, j\}$).
- y: Cualquier otro producto.
- m: Cualquier moneda valida (1, 2, 5, 10).
- s: Saldo actual.

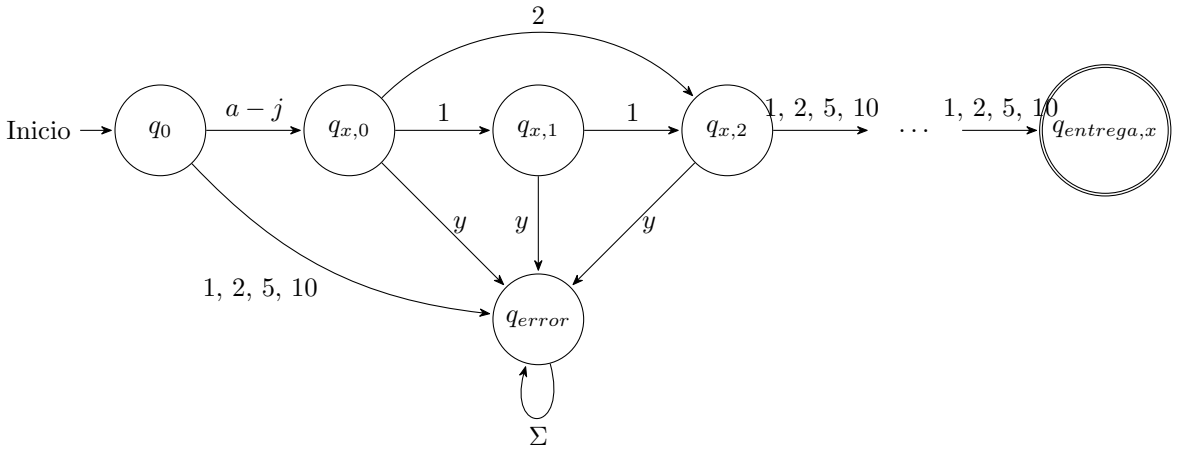
Q	**I** (x)	**I 1**	**I 2**	**I 5**	**I 10**	(y)
$\to q_0$	$q_{x,0}$	q_{error}	q_{error}	q_{error}	q_{error}	q_{error}
$q_{x,s}$	q_{error}	$q_{x,s+1}$	$q_{x,s+2}$	$q_{x,s+5}$	$q_{x,s+10}$	q_{error}
q_{error}	q_{error}	q_{error}	q_{error}	q_{error}	q_{error}	q_{error}

Table 1: El estado de aceptación F se alcanza cuando $s \geq precio(x)$.

Transiciones:

- **Validas:** Desde el estado inicial q_0, únicamente se acepta una letra ($a-j$). Una vez en el estado de acumulación $q_{x,s}$, solo se aceptan números.
- **Invalidas:** Si en q_0 se ingresa un número, se transita a q_{error}
  Si durante $q_{x,s}$ se intenta seleccionar otra letra, se transita a q_{error}.

4 Diagrama de estados



5 Validación de cadenas

5.1 Cadenas Válidas (Aceptadas)

1. **Cadena:** `g, 1, 2` (agua: $3)
   - **Camino:** $q_0 \xrightarrow{g} q_{agua,0} \xrightarrow{1} q_{agua,1} \xrightarrow{2} q_{agua,3}$
2. **Cadena:** `a, 5, 5` (cocacola: $10)
   - **Camino:** $q_0 \xrightarrow{a} q_{cocacola,0} \xrightarrow{5} q_{cocacola,5} \xrightarrow{5} q_{cocacola,10}$
3. **Cadena:** `c, 10, 10, 5` (sprite: $20)
   - **Camino:** $q_0 \xrightarrow{c} q_{sprite,0} \xrightarrow{10} q_{sprite,10} \xrightarrow{10} q_{sprite,20} \xrightarrow{5} q_{sprite,25}$
4. **Cadena:** `j, 10, 10, 10` (topochico: $30)
   - **Camino:** $q_0 \xrightarrow{j} q_{topochico,0} \xrightarrow{10} q_{topochico,10} \xrightarrow{10} q_{topochico,20} \xrightarrow{10} q_{topochico,30}$
5. **Cadena:** `d, 5` (squirt: $4)
   - **Camino:** $q_0 \xrightarrow{d} q_{squirt,0} \xrightarrow{5} q_{squirt,5}$

5.2 Cadenas Inválidas (Rechazadas)

1. **Cadena:** `1, 2, a`
   - **Camino:** $q_0 \xrightarrow{1} q_{error} \xrightarrow{2} q_{error} \xrightarrow{a} q_{error}$
2. **Cadena:** `b, c, 10`

- **Camino:** $q_0 \xrightarrow{b} q_{fanta,0} \xrightarrow{c} q_{error} \xrightarrow{10} q_{error}$

3. **Cadena:** `f, 10, 5` (pepsi: $17)
   - **Camino:** $q_0 \xrightarrow{f} q_{pepsi,0} \xrightarrow{10} q_{pepsi,10} \xrightarrow{5} q_{pepsi,15}$
4. **Cadena:** `g, g`
   - **Camino:** $q_0 \xrightarrow{g} q_{agua,0} \xrightarrow{g} q_{error}$
5. **Cadena:** `h, 10, 10, 5, 1, 1` (sangria: $28)
   - **Camino:** $q_0 \xrightarrow{h} q_{sangria,0} \xrightarrow{10,10,5,1,1} q_{sangria,27}$

6 Implementación

El modelo matemático se implementó en C

```c
#include <stdio.h>
#include <string.h>

typedef struct {
  char code;
  char product[15];
  int price;
} Products;

int main() {
  printf("=======================================\n");
  printf("   MAQUINA EXPENDEDORA DE REFRESCOS  \n");
  printf("=======================================\n");

  // productos y precios
  Products machine[10] = {{'a', "cocacola", 10}, {'b', "fanta", 8},
                          {'c', "sprite", 20},   {'d', "squirt",
    4},
                          {'e', "sidral", 23},   {'f', "pepsi",
    17},
                          {'g', "agua", 3},      {'h', "sangria",
    28},
                          {'i', "jarritos", 11}, {'j', "topochico",
     30}};

  // Estados
  int estado_actual = 0;    // 0 = q0, 1 = qpi,s, 2 = aceptacion,
    -1 = qerror
  int saldo_acumulado = 0;  // s
  int precio_objetivo = 0;  // precio(pi)
  char producto_elegido[15] = "";  // pi
  int total_productos = sizeof(machine) / sizeof(machine[0]);

  printf("\n-- MENU --\n");
  for (int i = 0; i < total_productos; i++) {
    printf(" [%c] %-10s | $%i\n", machine[i].code, machine[i].
    product,
```

```
32            machine[i].price);
33    }
34    printf("---------------------------------------\n");
35
36    do {
37      // salidas MOORE
38      if (estado_actual == 0) {
39        printf("\n[ESTADO q0]: Esperando seleccion.\n");
40        printf("-> Seleccione un producto (a - j): ");
41      } else if (estado_actual == 1) {
42        int faltante = precio_objetivo - saldo_acumulado;
43        printf("\n[ESTADO qpi,s]: Seleccion: %s | Saldo: $%i | Faltan
      : $%i\n",
44               producto_elegido, saldo_acumulado, faltante);
45        printf("-> Ingrese moneda (1, 2, 5, 10): ");
46      }
47
48      char entrada[20];
49      if (scanf("%19s", entrada) != 1) {
50        printf("Error.\n");
51        break;
52      }
53
54      // clasificar el simbolo
55      int es_producto = 0;
56      int es_moneda = 0;
57      int valor_moneda = 0;
58      char codigo_producto = '\0';
59
60      // producto o moneda
61      if (strlen(entrada) == 1 && entrada[0] >= 'a' && entrada[0] <=
      'j') {
62        es_producto = 1;
63        codigo_producto = entrada[0];
64      } else if (strcmp(entrada, "1") == 0) {
65        es_moneda = 1;
66        valor_moneda = 1;
67      } else if (strcmp(entrada, "2") == 0) {
68        es_moneda = 1;
69        valor_moneda = 2;
70      } else if (strcmp(entrada, "5") == 0) {
71        es_moneda = 1;
72        valor_moneda = 5;
73      } else if (strcmp(entrada, "10") == 0) {
74        es_moneda = 1;
75        valor_moneda = 10;
76      }
77
78      // Funcion de transicion
79      if (es_producto) {
80        if (estado_actual == 0) {
81          // q0 -> qpi,0
82          int indice = codigo_producto - 'a';
83          strcpy(producto_elegido, machine[indice].product);
84          precio_objetivo = machine[indice].price;
85          estado_actual = 1;
86        } else if (estado_actual == 1) {
```

```
87          // solo se permite una seleccion
88          printf("\nERROR:doble seleccion.\n");
89          estado_actual = -1;
90        }
91      } else if (es_moneda) {
92        if (estado_actual == 0) {
93          // no se puede insertar dinero antes de seleccionar un
    producto
94          printf("\nERROR No se puede insertar dinero primero.\n");
95          estado_actual = -1;
96        } else if (estado_actual == 1) {
97          // qpi,s --> qpi,s+m
98          saldo_acumulado += valor_moneda;
99
100         // aceptacion (s >= precio)
101         if (saldo_acumulado >= precio_objetivo) {
102           estado_actual = 2;
103         }
104       }
105     } else {
106       // simbolo que no esta en el alfabeto
107       printf("\n[!] Entrada no reconocida.\n");
108     }
109
110   } while (estado_actual != -1 && estado_actual != 2);
111
112   // aceptacion o Rechazo
113   printf("\n========================================\n");
114   if (estado_actual == 2) {
115     int cambio = saldo_acumulado - precio_objetivo;
116     printf(":) Transaccion Exitosa! (Estado Final)\n");
117     printf("Entregando producto: %s\n", producto_elegido);
118     if (cambio > 0) {
119       // si el dinero es mayor da cambio"
120       printf("Su cambio es: $%i\n", cambio);
121     }
122   } else if (estado_actual == -1) {
123     printf(":( Operacion Cancelada (Estado error).\n");
124     if (saldo_acumulado > 0) {
125       printf(" Devolviendo dinero: $%i\n", saldo_acumulado);
126     }
127   }
128   printf("========================================\n");
129
130   return 0;
131 }
```

Listing 1: Código fuente en C de la Máquina Expendedora

7 Conclusión

Esta actividad me permitio aplicar los conocimientos acerca de maquinas de estado, sus transiciones y estados, automatas, tablas de transicion, diagrama de estados, y arquitectura moore. Si soy honesto, se me dificulta comprender e intrepretar la definicion matematica, sin embargo el ejemplo que hice sobre una

maquina dispensadora de productos, me dejo entender mas, por lo que considero satisfactorio el resultado obtenido.

8 Referencias bibliográficas

Gómez Andrade, A. (2018). *Teoría de la Computación: Máquinas abstractas y modelos matemáticos.* Universidad de Guadalajara.